

Code Smells

Mindustry

Realizado por:
Miguel Carmo nº 65871

Code Smells Report

AI Package - Mindustry Codebase

Code Smell 1: Long Method

Location

- **Package**: mindustry.ai

- **Class**: “ControlPathfinder.java”

- **Method**: “getPathPosition()” (lines 1089-1282)

- **Lines of Code**: 193 lines

Code Snippet

```
public boolean getPathPosition(Unit unit, Vec2 destination, Vec2 mainDestination, Vec2 out, @Nullable boolean[]
noResultFound){
    if(noResultFound != null) noResultFound[0] = false;

    // Extract raycasting logic
    if(isDirectPathAvailable(unit, destination)){
        out.set(destination);
        return true;
    }

    // Extract path request management
    PathRequest request = getOrCreatePathRequest(unit, destination);

    // Extract path following logic
    return followPath(request, unit, destination, out, noResultFound);
}

private boolean isDirectPathAvailable(Unit unit, Vec2 destination){
    // Extract raycast caching and validation (lines 1121-1136)
}

private PathRequest getOrCreatePathRequest(Unit unit, Vec2 destination){
    // Extract request creation logic (lines 1148-1276)
}

private boolean followPath(PathRequest request, Unit unit, Vec2 destination, Vec2 out, boolean[] noResultFound){
    // Extract tile traversal logic (lines 1152-1256)
}
```

Exact Location

`*SE2526_65871_67940_68087_68231_68264_68681/core/src/mindustry/ai/ControlPathfinder.java*`

- **Method**: `getPathPosition(Unit unit, Vec2 destination, Vec2 mainDestination, Vec2 out, @Nullable boolean[] noResultFound)`

- **Lines**: 1089-1282

Rationale

This method is a clear example of a Long Method smell because:

1. **Multiple Responsibilities**: The method handles:

- Raycasting validation
- Path request creation and management
- Field cache lookup and validation
- Tile traversal and path following
- Path recalculation triggering
- Error handling

2. **High Dependency Coupling**: The method depends on 25+ external classes/packages, making it tightly coupled to the system

3. **Hard to Maintain**: Changes to any dependency (e.g., `PathRequest`, `FieldCache`, `Tile`) risk breaking this method

Based on dependency metrics analysis:

- **25+ external dependencies**
  - Accesses fields from: `PathRequest.lastUpdateId`, `PathRequest.destination`, `FieldCache.fields`, `FieldCache.frontier`

Proposed Refactoring

Extract Method Pattern: Break down the method into smaller, focused methods.

Benefits:

- Reduced Coupling: Each extracted method has fewer dependencies (lower Ce)

- Better Testability: Methods with lower CBO are easier to mock and test

- Dependency Isolation: Dependencies are isolated to specific methods, reducing propagation

- Improved Maintainability: Changes to dependencies affect smaller, focused methods

Code Smell 2: God Class

- **Package**: mindustry.ai

- **Class**: “ControlPathfinder.java”

- **Total Lines**: 1631 lines

Code Snippet

```
public class ControlPathfinder implements Runnable{
    // 1631 lines of code
    // Multiple responsibilities:
    // - Cluster management
    // - Path request handling
    // - Flow field calculation
    // - A* pathfinding
    // - Raycasting
    // - Thread management
    // - Debug rendering
    // - Cache management
}
```

Exact Location
`SE2526_65871_67940_68087_68231_68264_68681/core/src/mindustry/ai/ControlPathfinder.java`

- **Class**: `ControlPathfinder`

- **Lines**: 1-1631

Rationale

This class is a God Class (also known as Blob or Large Class) because:

1. **Too Many Responsibilities**: The class handles:

  - Cluster-based pathfinding (`updateCluster`, `getCreateCluster`)

- Path request management (`PathRequest`, `initializePathRequest`)

- Flow field computation (`updateFields`, `addFlowCluster`)

- A pathfinding algorithms (`innerAstar`, `clusterAstar`)

- Raycasting utilities (`raycast`, `raycastFast`, `raycastRect`)

2. **Very High Efferent Coupling**: The class depends on 45+ external classes/packages, creating tight coupling

3. **Difficult to Test**: High coupling requires extensive mocking of 45+ dependencies for unit testing

4. **Dependency Propagation**: Changes in any of the 45+ dependencies can break this class

Related Dependency Metrics

Based on dependency metrics analysis:

- **Efferent Coupling** : 45+ external dependencies

Proposed Refactoring

Extract Class Pattern: Split the class into focused, cohesive classes:

Exemple

```
public class ClusterManager {
   private Cluster[][][] clusters;
   public Cluster getCreateCluster(int team, int pathCost, int cx, int cy) { }
   public void updateCluster(int team, int pathCost, int cx, int cy) { }
   public void updateInnerEdges(int team, PathCost cost, int cx, int cy, Cluster cluster) {
}
}
```

Benefits:

- **Reduced Efferent Coupling**: Each class has fewer dependencies (Ce reduced from 45+ to ~8-12 per class)

- **Lower Instability**: Split classes have lower I values, making them more stable

- **Better Dependency Management**: Dependencies are isolated to specific classes

- **Improved Testability**: Classes with lower coupling require less mocking

Code Smell 3: Magic Numbers and TODO Comments

Location

- **Package**: `mindustry.ai`

- **Class**: `RtsAI.java`

- **Lines**: Multiple locations throughout the file

Code Snippet

```
public class ClusterManager {
    private Cluster[][][] clusters;
    public Cluster getCreateCluster(int team, int pathCost, int cx, int cy) { }
    public void updateCluster(int team, int pathCost, int cx, int cy) { }
    public void updateInnerEdges(int team, PathCost cost, int cx, int cy, Cluster cluster) { }
}

// 2. Path Request Manager
public class PathRequestManager {
    private ObjectMap<Unit, PathRequest> unitRequests;
    public PathRequest getOrCreateRequest(Unit unit, Vec2 destination) { }
    public void invalidateRequest(PathRequest request) { }
}

// 3. Flow Field Calculator
public class FlowFieldCalculator {
    public void updateFields(FieldCache cache, long nsToRun) { }
    public void addFlowCluster(FieldCache cache, int cx, int cy, boolean addingFrontier) { }
}

// 4. Pathfinding Algorithm
public class ClusterPathfinder {
    public @Nullable IntSeq clusterAstar(PathRequest request, int pathCost, int startNodeIndex, int endNodeIndex) { }
    public float innerAstar(int team, PathCost cost, int minX, int minY, int maxX, int maxY, ...) { }
}

// 5. Raycast Utilities
public class PathRaycast {
    public static boolean raycast(int team, PathCost type, int x1, int y1, int x2, int y2) { }
    public static int raycastFast(int team, PathCost type, int x1, int y1, int x2, int y2) { }
}

// 6. Main Coordinator
public class ControlPathfinder implements Runnable {
    private ClusterManager clusterManager;
    private PathRequestManager requestManager;
    private FlowFieldCalculator flowCalculator;
    private ClusterPathfinder pathfinder;
    private PathRaycast raycast;

    public boolean getPathPosition(Unit unit, Vec2 destination, Vec2 out, boolean[] noResultFound) {
        // Coordinate between components
    }
}
```

Exact Location

- **File**: SE2526_65871_67940_68087_68231_68264_68681/core/src/mindustry/ai/RtsAI.java

- **Lines with Magic Numbers**: 31, 32, 81, 1000, 350, etc.

- **Lines with TODO Comments**: 60, 87, 171, 190, 327, etc.

Rationale

This code smell encompasses two related issues:

3.1 Magic Numbers

Problems:

1. **Lack of Meaning**: Numbers like `60f`, `15`, `1000f` have no clear meaning without context

2. **Hard to Maintain**: Changing these values requires finding all occurrences

3. **No Documentation**: The reasoning behind specific values is not documented

Examples from code:

- `squadRadius = 60f` - Why 60? What units?

- `maxTargetsChecked = 15` - Why 15? Should this be configurable?

- `checkRange = 350f` - Why 350? What does this represent?

3.2 TODO Comments

Problems:

1. **Technical Debt**: TODOs indicate incomplete or problematic code

2. **Maintenance Issues**: Code marked as "atrocious" or "extremely slow" remains in production

3. **Lack of Documentation**: TODOs often lack context or resolution plans

Examples from code:

- `//TODO atrocious implementation` - Indicates code that should be refactored

- `//TODO extremely slow especially with many squads` - Performance issue not addressed

- `//TODO: very dangerous usage` - Potential concurrency bug

Related Dependency Metrics

Based on dependency metrics analysis:

- Number of Dependencies (NOD): 20+ classes/packages

Proposed Refactoring

For TODO Comments- Create Issues and Refactor:

// Replace TODO comments with:

// 1. Extract to separate methods/classes

// 2. Add proper documentation

// 3. Create performance optimizations

// 4. Fix concurrency issues

Benefits:

- **Reduced Hidden Dependencies**: Constants make dependencies on configuration values explicit

- Better Dependency Management: TODOs related to dependencies (e.g., thread safety) are addressed

- Reduced Static Coupling: Eliminates dependencies on global state where possible

Summary

The identified code smells, analyzed through dependency metrics, indicate areas where the codebase could benefit from refactoring:

1. **Long Method**

2. **God Class**

3. **Magic Numbers & TODOs**